DATA ANALYTICS INTERNSHIP

Assignment 2: Python, NumPy, Pandas, Data Analysis & Visualization

Setup Requirement

Set up the following before starting:

- Python 3.x with **Jupyter Notebook** (or use Google Colab which needs no install).
- Install libraries: **pip install numpy pandas matplotlib seaborn**

Task 1: Python Basics & NumPy Operations

1.1 Python Basics

Write a single Python program that demonstrates Python data types, variables, and operators. The program must:

- Declare one variable each of type **int, float, string, bool, list, tuple, dict**. Print each value and its type.
- Take 2 numbers and demonstrate all **arithmetic operators** (+, -, *, /, %, **, //).
- Demonstrate any 3 **comparison operators** (==, >, <) and any 2 **logical operators** (and, or).

```
# Data Types
i = 25
f = 3.14
s = "Data Analytics"
b = True
lst = [10, 20, 30]
tup = (1, 2, 3)
dct = {"name": "Aarav", "age": 28}

print(i, type(i))
print(f, type(f))
print(s, type(s))
# (continue for the rest)

# Arithmetic
a, b_num = 10, 3
print("Add:", a + b_num)
print("Sub:", a - b_num)
print("Mul:", a * b_num)
print("Div:", a / b_num)
print("Mod:", a % b_num)
print("Power:", a ** b_num)
print("Floor div:", a // b_num)
```

1.2 NumPy Operations

Use the following data for the NumPy exercises:

Monthly sales of an electronics store (in INR thousands) for the last 12 months:
[125, 138, 142, 155, 168, 180, 195, 210, 225, 240, 258, 275]

Perform the following operations:

- Create a NumPy array **sales** from the above data.
- Print the **shape, size, and dtype** of the array.
- Calculate and print: **sum, mean, max, min, standard deviation**.
- Reshape the array into a **4 rows x 3 columns** 2D array. Print the result.

• Find the indices of months where sales were **greater than 200**.

• Calculate the **month-over-month growth** using **np.diff(sales)** and print it.

```
import numpy as np

sales = np.array([125, 138, 142, 155, 168, 180,
                  195, 210, 225, 240, 258, 275])

print("Shape:", sales.shape)
print("Size:", sales.size)
print("Dtype:", sales.dtype)
print("Sum:", sales.sum())
print("Mean:", sales.mean())
print("Max:", sales.max())
print("Min:", sales.min())
print("Std:", sales.std())

reshaped = sales.reshape(4, 3)
print("Reshaped (4x3):")
print(reshaped)

high_sales_idx = np.where(sales > 200)
print("Months with sales > 200:", high_sales_idx)

growth = np.diff(sales)
print("Month-over-month growth:", growth)
```

Task 2: Pandas - Data Loading, Cleaning & Merging

Dataset 1: Customers (Save as customers.csv or create using a dict)

cust_id	name	city	age	gender
C01	Aarav	Mumbai	28	M
C02	Bhavya	Delhi	32	F
C03	Chirag	Bangalore	26	M
C04	Diya	Mumbai	NaN	F
C05	Eshan	Pune	30	M
C06	Farah	Delhi	29	F
C07	Gaurav	Bangalore	40	M
C08	Hina	Mumbai	27	NaN

Dataset 2: Purchases (Save as purchases.csv)

purchase_id	cust_id	product	amount	purchase_date
P101	C01	Laptop	55000	2025-02-03
P102	C02	Saree	4500	2025-02-05
P103	C03	Smartphone	32000	2025-02-09
P104	C01	Tablet	18000	2025-02-11
P105	C05	Headphones	1500	2025-02-12
P106	C02	Watch	3500	2025-02-14
P107	C06	T-shirt	499	2025-02-18
P108	C07	Backpack	1800	2025-02-20

P109	C03	Mouse	899	2025-02-22
P110	C05	Charger	650	2025-02-26

2.1 Loading and Inspection

- Load both datasets into Pandas DataFrames named **customers_df** and **purchases_df**.
- Display the first 5 rows of each: **customers_df.head()** and **purchases_df.head()**.
- Print the shape and column data types of both: **df.info()**.

2.2 Data Cleaning

On the customers DataFrame, perform:

- Identify missing values using **customers_df.isnull().sum()**.
- Fill missing **age** values with the **mean age**.
- Fill missing **gender** with the value **'Unknown'**.
- Convert **age** column to integer type.

2.3 Data Merging

Merge the two DataFrames using **cust_id** as the key:

```
import pandas as pd

merged_df = pd.merge(customers_df, purchases_df,
                     on='cust_id', how='inner')
print(merged_df.head())
print("Merged shape:", merged_df.shape)
```

Submit the head() of the merged DataFrame and its shape.

Task 3: Filtering, Grouping, Aggregation & Data Visualization

Continue using the **merged_df** from Task 2.

3.1 Filtering

- Show all purchases from customers in **Mumbai**.
- Show all purchases with **amount > 5000**.
- Show all male customers (gender = 'M') who made any purchase.

3.2 Grouping & Aggregation

Perform the following groupby operations and print results:

- **Total purchase amount per city** (groupby city, sum of amount).
- **Average purchase amount per gender**.
- **Number of purchases per customer** (groupby cust_id, count).
- **Top 3 customers by total spending**.

```
# Total by city
print(merged_df.groupby('city')['amount'].sum())

# Average by gender
print(merged_df.groupby('gender')['amount'].mean())

# Number of purchases per customer
print(merged_df.groupby('cust_id').size())

# Top 3 customers by total spending
top3 = merged_df.groupby('name')['amount'].sum().sort_values(ascending=False).head(3)
```

```
print(top3)
```

3.3 Data Visualization

Create the following 4 plots using **matplotlib** or **seaborn**:

#	Plot Type	What to Plot
1	Bar Chart	Total purchase amount per city
2	Pie Chart	Share of purchases by gender
3	Bar Chart (Seaborn)	Top 3 customers by total spending
4	Line Chart	Purchase date vs amount (sorted chronologically)

Each plot must have a clear **title**, **X-axis label**, **Y-axis label**, and appropriate colours.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Bar chart: total by city
city_totals = merged_df.groupby('city')['amount'].sum()
plt.figure(figsize=(7, 4))
city_totals.plot(kind='bar', color='steelblue')
plt.title('Total Purchase Amount by City')
plt.xlabel('City')
plt.ylabel('Total Amount (INR)')
plt.tight_layout()
plt.show()
```

3.4 Data Analysis Summary (Short Conclusion)

Write a short 100 to 150 word conclusion based on your analysis. Cover: highest revenue city, most active customer, average spending pattern, and any other interesting insight you discovered.

Deliverables

- Task 1: Python data types + arithmetic / comparison / logical operator program output, and NumPy operations output for the 12-month sales data.
- Task 2: Customers and Purchases DataFrames loaded, cleaned (missing values handled), and successfully merged. Submit code and screenshots.
- Task 3: Filtering queries, groupby aggregations (4 outputs), 4 visualizations, and a 100 to 150 word conclusion paragraph.

How to Submit

1. Create the assignment in MS Word (.doc / .docx) or PDF format.
2. Include your full name, course / internship name, and assignment number at the top of the document.
3. Paste your Python code and clear screenshots of every output for each task.
4. Upload the completed file to your Google Drive.
5. Enable sharing access by setting the file to "Anyone with the link can view".
6. Copy the Google Drive link and submit it before the assignment deadline through the official submission portal.